BRIEFING DO PROJETO: LPFIN (Controle Financeiro Pessoal)

1. Visão Geral e Contexto

O lpfin é uma aplicação web responsiva (desktop e mobile) para controle financeiro pessoal. O objetivo é substituir uma planilha complexa do Google Sheets (que será anexada como referência visual) por um sistema automatizado.

O grande diferencial deste sistema é a projeção de cenários futuros baseada em parcelamentos e contas recorrentes.

2. Stack Tecnológica

- **Frontend/Backend**: Next.js (App Router), React, TypeScript, Tailwind CSS.
- **Banco de Dados e Autenticação**: Supabase (PostgreSQL).
- **Gráficos**: Recharts (ou similar).
- **Hospedagem**: Vercel.
- **Versionamento**: Git/GitHub (repositório já configurado e vinculado ao Source Control do VS Code).

3. Configurações de Ambiente (Supabase)

O projeto usará as seguintes variáveis de ambiente. O Cline deve criar o arquivo .env.local e adicionar essas variáveis, garantindo que o .env.local esteja no .gitignore.

- **NEXT_PUBLIC_SUPABASE_URL** = https://hyubceacbustqjhmlovb.supabase.co
- **NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY** = sb_publishable_9HkpDHZh96OnkP25wjKOKw_xsgu0PXX
- **SUPABASE_SECRET_KEY** = [INSERIR A CHAVE SECRETA AQUI APENAS NO .env.local] (Uso restrito ao backend, nunca expor no frontend ou commitar)
- **SUPABASE_JWKS_URL** = https://hyubceacbustqjhmlovb.supabase.co/auth/v1/.well-known/jwks.json

4. Arquitetura e Estrutura de Dados

O banco de dados deve ser relacional e a hierarquia de dados baseia-se em: Usuário -> Linha do Tempo (Ano/Mês) -> Categorias de Contas -> Lançamentos. O "Mês" não é uma tabela mãe, mas sim a chave de referência (Mês/Ano) que une os dados no painel do usuário.

4.1. Entidades e Relacionamentos:

- **Users (Usuários)**: Raiz do sistema. Gerenciado pelo Supabase Auth.
- **Cards (Cartões)**: Pertencem ao Usuário. (Nome, limite, fechamento, vencimento).
- **Fixed_Bills (Cadastro de Contas Fixas)**: Pertencem ao Usuário. (Nome da conta, dia de vencimento). É apenas o "molde" da conta.
    - 3.1. **Fixed_Bills_Entries (Lançamentos Mensais de Contas Fixas)**: Relaciona a Conta Fixa ao Mês/Ano de referência (reference_year, reference_month) e armazena o valor real daquele mês. (Relacionamento 1:N com Fixed_Bills).
- **Incomes (Ganhos/Receitas)**: Pertencem ao Usuário. Possuem Mês/Ano de referência e valor.
- **Transactions (Compras/Transações)**: Pertencem ao Usuário e opcionalmente a um Cartão. (Descrição, valor total, mês/ano de início, categoria).
    - 5.1. **Installments (Parcelas)**: Tabela gerada a partir de uma Transaction. É aqui que o Mês/Ano vive de fato. (ID da transação, número da parcela ex: 1/3, mês/ano de referência, valor editável).
- **Payroll_Deductions (Débitos em Folha)**: Pertencem ao Usuário. Podem ser fixos ou parcelados. (Nome, valor, mês/ano de referência).

Resumo da Lógica de Dados: *Para renderizar a tela do mês de "Outubro/2026" de um usuário, o sistema faz uma query buscando todos os registros que tenham reference_year = 2026 e reference_month = 10 nas tabelas Fixed_Bills_Entries, Incomes, Installments e Payroll_Deductions, filtrando pelo user_id.*

5. Requisitos Funcionais (Telas e Funcionalidades)

5.1. Autenticação

- Login e cadastro de usuários.
- **Obrigatório**: Botão "Login com Google" (OAuth via Supabase).

5.2. Página Inicial (Visão Mensal)

- Deve exibir por padrão o mês e ano atual, com seletor para navegar entre meses/anos.
- Apresentação do Mês Selecionado:
    1. **Cartões**: Compras ativas no mês, divididas por cartão.
    2. **Contas Fixas**: Contas do mês, com campo para o usuário editar o valor real daquele mês específico.
    3. **Débitos em Folha**: Débitos do mês.
    4. **Ganhos**: Total de receitas do mês.

5. **Balanço Mensal**: Cálculo (Ganhos) - (Cartões + Contas Fixas Reais + Débitos em Folha). Exibir em verde se positivo e vermelho se negativo.

5.3. Regra de Negócio: Motor de Parcelamento (CRÍTICO)

- Compra parcelada (ex: 3x de R$ 100 em Out/2026) gera registros automáticos na tabela installments.
- O status avança automaticamente conforme o mês corrente.
- **Ajuste de Valor Individual**: O usuário pode editar o valor de uma parcela específica (ex: a primeira parcela veio R$120 e as outras duas R$90), sem afetar as demais ou o valor total da transação.

5.4. Dashboard e Previsão Futura

- **Gráfico 1 (Gastos Mensais)**: Histórico dos últimos 6 meses.
- **Gráfico 2 (Previsão Futura)**: Projeção para os próximos 6 meses a 1 ano.
- Lógica da Previsão Futura:
    - **Contas Fixas**: Média histórica real dos valores inputados (últimos 3 a 6 meses). Se não houver histórico, usar o valor base.
    - **Parcelas**: Soma exata dos valores das parcelas já geradas para os meses futuros (respeitando ajustes individuais).
    - **Categoria "Outros" (Cartão)**: Média histórica da categoria projetada para o futuro.

6. Instruções de Desenvolvimento para o Agente (Cline)

- **Abordagem Passo a Passo**: Construir por etapas (Setup -> Auth -> Banco de Dados -> CRUD -> Visão Mensal -> Dashboard).
- **Git e Commits**: O repositório local já está inicializado, vinculado ao GitHub (origin) e configurado no Source Control do VS Code. Portanto, NÃO execute git init ou git remote add. Ao final de cada etapa concluída com sucesso, faça o git add ., git commit -m "mensagem descritiva" e git push utilizando o terminal.
- **Código Limpo**: TypeScript estrito, componentes funcionais, lógica financeira isolada em /lib/finance.ts.
- **Mobile-first**: Layout pensado para celular e adaptado para desktop.
- **Atenção à Arquitetura**: Tabela installments para edição individual de parcelas e fixed_bills_entries para contas variáveis.